Exercise 8

A walker that knows it's grounded

At the end, the walker knows when it stands on the ground, through a sensor at its feet.

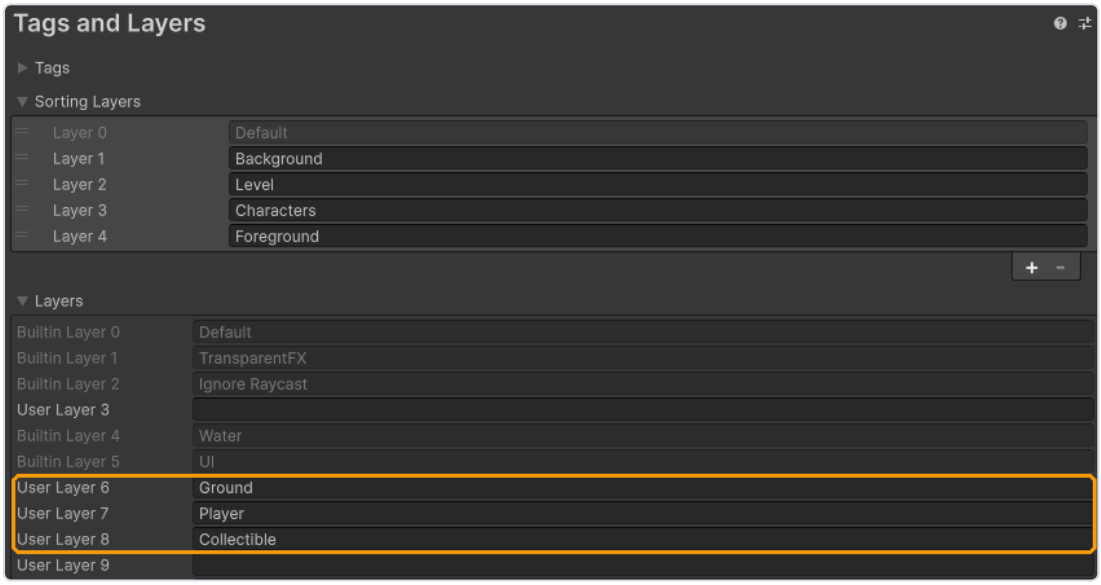
Before you start

- Exercise 7 done and pushed: the walker slides down walls. In a new session, run `git pull` first

1 Physics layers

Physics layers are a second list, separate from the sorting layers: they tell physics queries what to look at, and the ground sensor will look only at `Ground`.

- **Edit** › **Project Settings** › **Tags and Layers**, expand **Layers** and type `Ground`, `Player` and `Collectible` into User Layer 6, 7 and 8; `Collectible` is for the coins of Exercise 24.



Physics layers go in Layers, under the Sorting Layers from Exercise 2.

- Select `Ground` (under `Grid`) and set **Layer**, at the top right of the Inspector beside Tag, to `Ground`. It is not the Sorting Layer, which only decides what draws in front.
- Select `Walker` and set its Layer to `Player`. Nothing changes on screen: a layer matters only when something asks about it.

2 The sensor's anchor

The ground sensor needs a point to ask about: an empty child at the walker's feet, which moves with it.

- Right-click `Walker` in the Hierarchy, **Create Empty**, and name the child `GroundCheck`; it takes the walker's `Player` layer.
- Set its Position to `(0, -0.05, 0)`, relative to the walker: just under the feet, where the collider's bottom edge is.

3 Script, run 1: settings and a visible sensor

The script grows in six runs, over this exercise and the next three; press Play after each one, so a mistake can only be in the lines you just added. Run 1 adds the jump's settings and draws the sensor, so you can check its

place before any code uses it.

Open `WalkerController` and, below the Movement block, add:

```
    [Header("Jump")]
    [SerializeField] private float jumpVelocity = 15f;
    [SerializeField] private Transform groundCheck;
    [SerializeField] private float groundCheckRadius = 0.15f;
    [SerializeField] private LayerMask groundLayer;

    private InputAction jumpAction;
    private bool jumpRequested;
    private bool isGrounded;
```

In `Awake`, after the `moveAction` line, add:

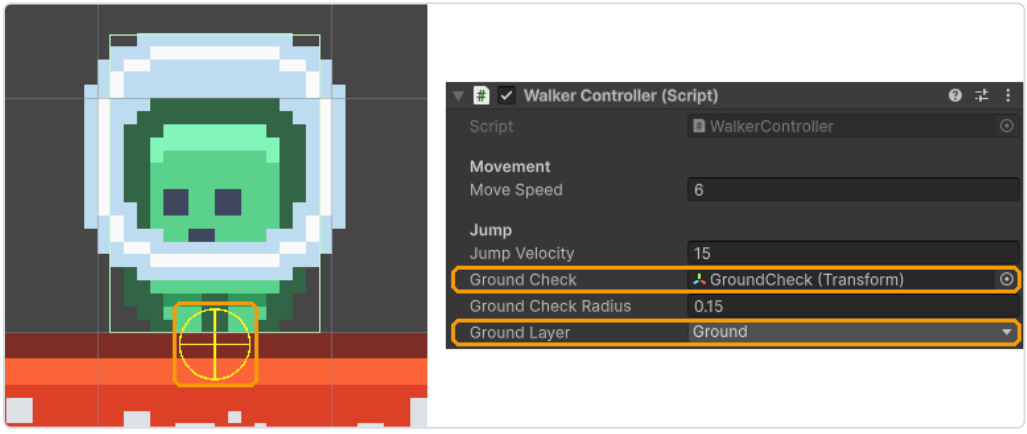
```
        jumpAction = InputSystem.actions.FindAction("Jump");
```

At the bottom of the class, add the gizmo; the `null` check matters because gizmos also draw in Edit mode, before the slot is filled:

```
    void OnDrawGizmosSelected()
    {
        if (groundCheck == null) return;

        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(groundCheck.position, groundCheckRadius);
    }
```

- Save and go back to Unity: one yellow warning says `jumpVelocity` is never used, which Run 3, in Exercise 9, fixes.
- Select `Walker`, drag `GroundCheck` from the Hierarchy into Walker Controller, Ground Check, and set Ground Layer to `Ground` only.
- A yellow circle appears at the feet, overlapping the ledge's top edge; if it doesn't reach the ledge, move `GroundCheck` until it does.



The circle sits just under the collider's green edge and overlaps the ledge.

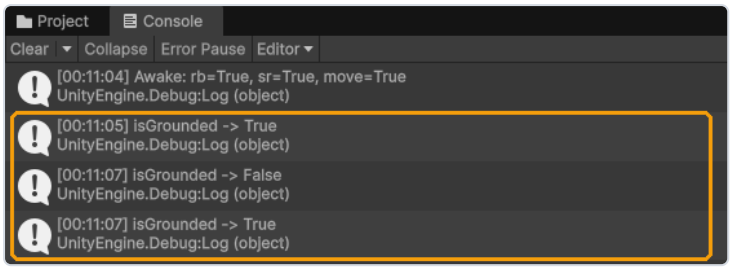
4 Run 2: sense the ground

Run 2 asks the physics engine, every physics step, whether anything on the Ground layer overlaps the circle, and logs only the changes.

At the top of `FixedUpdate`, add:

```
        bool wasGrounded = isGrounded;
        isGrounded = Physics2D.OverlapCircle(
            groundCheck.position, groundCheckRadius, groundLayer) != null;
        if (isGrounded != wasGrounded) Debug.Log($"isGrounded -> {isGrounded}");
```

- **Play** and walk left off the ledge: the Console shows `isGrounded -> True` at the start, `False` as you leave the ledge and `True` as you land. Stop.



One line per change, not one per physics step.

- Always `False`: the circle misses the ground, or `Ground` is not on the Ground layer. Always `True`: Ground Layer also includes `Player`, so the sensor sees the walker itself.
- Delete the `wasGrounded` line and the `Debug.Log` line; keep the `OverlapCircle` line.
- `Ctrl` + `S`, then commit and push with the message `Add physics layers and a ground check`.